

NearAid — Repository Evidence Report

Kotlin Multiplatform mutual-aid app (Android + iOS) · master-resume attribution audit
Generated 2026-07-20 · branch **KMP** (post CI/quality merge) · every figure is drawn directly from the repo; coverage and performance are measured, nothing is inferred.

18

GRADLE MODULES

43

COMMITTS (ALL BRANCHES)

1

CONTRIBUTOR (SOLO)

18%

LINE COVERAGE (KOVER)

585ms

COLD START (EMULATOR)

1 Module structure

18 modules — NIA-style multi-module layout with a **build-logic** included build providing convention plugins (`nearaid.android.*`, `nearaid.kmp.*`, `nearaid.cmp.library`, `nearaid.jvm.library`, `nearaid.quality`, `nearaid.kover`). Source: `settings.gradle.kts` .

GROUP	MODULES
Host (2)	<code>:app</code> (Android Activity, thin) · <code>:shared</code> (iOS umbrella framework, exports to Xcode as <code>Shared</code>)
Core (9)	<code>:core:common</code> · <code>:core:model</code> · <code>:core:designsystem</code> · <code>:core:navigation</code> · <code>:core:datastore</code> · <code>:core:network</code> · <code>:core:database</code> · <code>:core:domain</code> · <code>:core:data</code>
Feature (6)	<code>:feature:auth</code> · <code>:feature:discovery</code> · <code>:feature:post</code> · <code>:feature:activity</code> · <code>:feature:messages</code> · <code>:feature:profile</code>
Performance (1)	<code>:benchmark</code> (AndroidX Macrobenchmark — cold/warm startup; added during this audit, see §5)

2 Full stack — from `libs.versions.toml`

CATEGORY	TECHNOLOGY
Language / platform	Kotlin 2.1.20 Multiplatform (Android + iOS) · Compose Multiplatform 1.7.3 · AGP 8.7.3 · Java 17 · KSP 2.1.20-1.0.32 (KSP1)
UI	Jetbrains Compose (shared UI), Material3, Navigation Compose, Coil 2/3
Architecture	MVI + Clean Architecture · multi-module + convention plugins
DI	Koin 4.0.3 (migrated off Hilt — commit <code>a309abc</code>)
Networking	Ktor 3.0.3 client (OkHttp + Darwin engines), WebSockets, content-negotiation, auth, logging, mock · <code>kotlinx.serialization</code> 1.7.3
Async	Coroutines 1.9.0 + Flow · <code>kotlinx-datetime</code> 0.6.1
DB / persistence	Room-KMP 2.7.1 (bundled SQLite 2.5.1) · DataStore 1.1.1 · <code>androidx.security-crypto</code> (secure tokens) · Okio
iOS interop	SKIE 0.10.4
Push	Firebase BOM 33.7.0 / messaging <code>wired, pending config</code>
Testing	JUnit4 · <code>androidx-junit</code> · Espresso · Robolectric 4.14.1 · MockK 1.13.13 · Turbine 1.2.0 · <code>coroutines-test</code>
Quality / coverage	<code>detekt</code> 1.23.7 · <code>Spotless</code> 6.25.0 / <code>ktlint</code> 1.3.1 · Kover 0.9.1 (coverage) · Macrobenchmark 1.3.3 + <code>UiAutomator</code> 2.3.0 (startup)
Debug tooling	LeakCanary 2.14
CI / CD	GitHub Actions — <code>ci.yml</code> (unit tests, lint, <code>assembleDebug</code> , <code>detekt</code> , <code>Spotless</code> , iOS shared tests + Xcode build on macOS runner) · <code>release.yml</code> (signed APK → Firebase App Distribution on <code>v*</code> tags)

3 What the app does & what shipped

NearAid (পাশের মানুষ, *"the person beside you"*) — a hyperlocal **mutual-aid** app connecting people who need everyday help (food, clothes, medicine, shelter) with nearby neighbours. A two-sided board (Seeker/Helper, Giver/Recipient). No money changes hands — it is a discovery + coordination layer; handoff happens offline. **One shared Kotlin + Compose codebase runs on both Android and iOS.**

Shipped (README features + commit history):

- Phone/OTP auth + profile setup
 - Two-sided discovery (Needs/Offers, filters, banded/fuzzed distance)
 - Post request / offer flow
 - Claim → chat → deliver → confirm → rate
 - Realtime 1:1 chat over WebSocket
- Messages tab, profile / trust / ratings
 - Report + block safety
 - Dark theme (semantic color system)
 - Accessibility (TalkBack / VoiceOver)
 - Bangla / English localization
 - Shared UI + logic on Android & iOS

Deferred / not fully shipped (state honestly):

- `deferred` Map view — replaced by list-based discovery in v1
- `pending` FCM push — wired but pending Firebase config
- `deferred` iOS release CD — pending Apple Developer account (commit `1e2d33d`)

4 Commit attribution

```
# requested command
$ git log --author="alfarukemail@gmail.com" --oneline | wc -l    → 36  (this branch)
$ git shortlog -sne --all                                       → 43  faruk <alfarukemail@gmail.com>
```

100% solo repository. Every commit on every branch (`main` , `KMP` , `ci/setup-cicd`) is yours — **43 of 43 commits, single author.** There are no team commits.

ARC	COVERAGE
Foundation	First commit, gitignore, unit tests (app / core / features)
Android polish	Dark theme, accessibility, string resources
KMP migration <code>largest arc</code>	Phase 0 skeleton → Hilt→Koin → pure core to commonMain → Retrofit→Ktor + DataStore-KMP → Room-KMP + <code>:core:data</code> → ViewModels to commonMain → iOS drives shared ViewModels (SKIE) → Compose Multiplatform shared UI → platform edges (Coil, iOS picker, secure tokens, shared a11y)
Chat / realtime	WebSocket failure handling, DTO field mapping fixes
CI / CD	PR validation, <code>detekt</code> + <code>Spotless</code> , Android release CD

Attribution guidance: claim full individual ownership. Frame as *"sole developer / individual project"* — **do not use** "led a team" or "collaborated"; there is no evidence of other contributors.

5 Metric evidence

METRIC	STATUS	WHAT THE REPO ACTUALLY CONTAINS
--------	--------	---------------------------------

Test coverage %	now evidenced	18.0% overall line coverage (1,176 / 6,541 lines) — measured with Kotlinx Kover 0.9.1, wired as a <code>nearaid.kover</code> convention plugin aggregating all modules over the JVM/Android unit tests (<code>./gradlew koverXmlReport</code>). Also 13.9% branch, 21.3% method, 34.1% class. Composables and generated code are excluded so the figure reflects testable logic. Coverage is uneven — see breakdown below. Cite as “~18% line coverage (Kover), concentrated in ViewModels & core”, not as broad coverage.
Startup time	now evidenced	Cold start 585 ms median (531–758 ms), warm start 116 ms median (98–179 ms) — <code>timeToInitialDisplayMs</code> , 5 iterations each, AndroidX Macrobenchmark 1.3.3 (<code>StartupTimingMetric</code>) against a new profileable, non-minified <code>benchmark</code> app variant. Caveat: run on an Android emulator (API 37, <code>suppressErrors=EMULATOR</code>) — not representative of a specific physical device; cite as an emulator baseline or re-run on a phone. An <code>adb am start -W</code> cross-check on the debug build gave ~1.0–1.45 s (debug + LeakCanary overhead, expected higher).
Build time	measured, noisy	Gradle <code>—profile</code> reports generated (Gradle 8.11.1, 402 tasks). Configuration ~1.1–1.3 s; clean <code>:app:assembleDebug</code> ranged ~20 s to ~1m40 s wall-clock across runs depending on daemon/machine warmth, and a no-op re-ran 322/402 tasks (no configuration cache). Too variable to cite a single build-time figure honestly — report the range with the caveat, or stabilize (configuration cache + clean-room run) before quoting a number.

Coverage by module (line %, Kover merged report) — concentrated in ViewModels, the design system and core utilities; the data/persistence/model layers are effectively untested:

TIER	MODULES (LINE COVERAGE)
Tested 25–63%	<code>core:common</code> 62.9% · <code>core:designsystem</code> 41.2% · <code>feature:messages</code> 34.9% · <code>feature:discovery</code> 32.1% · <code>feature:activity</code> 31.1% · <code>feature:profile</code> 25.8%
Light 5–25%	<code>feature:auth</code> 24.5% · <code>feature:post</code> 24.3% · <code>core:network</code> 7.3% · <code>core:domain</code> 5.2%
Untested 0%	<code>core:database</code> · <code>core:data</code> · <code>core:model</code> · <code>core:datastore</code> · <code>core:navigation</code> · <code>app</code> · <code>shared</code>

6 Team-size signal

Solo No `CODEOWNERS` file exists. `git shortlog -sne --all` reports exactly one contributor (faruk). Every signal points to a single-developer project.

7 Resume honesty summary

SAFE TO CLAIM (EVIDENCED)
18-module KMP architecture · full stack above · sole authorship · CI/CD pipeline · WebSocket realtime chat · complete Android→KMP migration · shared Compose UI on Android + iOS · ~18% line coverage

8 Resume bullets — ready to paste

Every number below is backed by the evidence in this report. Pick 3–5; the emulator/coverage caveats are baked into the wording so the claims survive scrutiny.

Project header

NearAid — Kotlin Multiplatform mutual-aid app (Android + iOS) · Sole developer · Kotlin, Compose Multiplatform, Ktor, Koin, Room-KMP

Bullets (impact-first)

- Sole developer of a hyperlocal mutual-aid app spanning **18 Gradle modules** on a single **Kotlin Multiplatform** codebase — shared business logic and Compose Multiplatform UI render on both Android and iOS from one source (43/43 commits authored solo).
- Architected an **MVI + Clean Architecture** multi-module setup with custom **build-logic convention plugins**, keeping 15 feature/core modules on consistent, DRY build config.
- Led a full **Android→KMP migration**: swapped Hilt→**Koin**, Retrofit→**Ktor** (incl. WebSocket realtime chat), and moved persistence to **Room-KMP** + DataStore with platform-secure token storage.
- Built a **CI/CD pipeline** (GitHub Actions): unit tests, Android lint, detekt + Spotless gates, iOS shared-framework tests + Xcode build on a macOS runner, and signed-APK release to Firebase App Distribution on version tags.
- Instrumented quality & performance tooling: **~18% line coverage** (Kotlinx Kover, concentrated in ViewModels & core modules) and startup benchmarking via **AndroidX Macrobenchmark** — **~0.6 s cold / ~0.12 s warm** time-to-initial-display (emulator baseline).
- Shipped accessible, localized UI: TalkBack/VoiceOver semantics, a semantic light/dark color system, and Bangla/English string resources — validated by a shared cross-platform accessibility contract.

Tech-stack line

Kotlin Multiplatform · Compose Multiplatform · Coroutines/Flow · Ktor · Koin · Room-KMP · DataStore · kotlinx.serialization · SKIE · JUnit/MockK/Turbine/Robolectric · Kover · Macrobenchmark · detekt/Spotless · GitHub Actions · Gradle convention plugins

Keep out (unsupported by evidence): "high/comprehensive test coverage" (it's ~18%), any single build-time figure (measurements are noisy), device-specific startup numbers (only measured on emulator), and any "team/led a team" framing (solo project).

All figures verified against the working tree on branch `KMP` at generation time. Sources: `settings.gradle.kts`, `gradle/libs.versions.toml`, `.github/workflows/{ci,release}.yml`, `README.md`, `git log / shortlog`, Kover merged report (`koverXmlReport`), Gradle `—profile` reports, and AndroidX Macrobenchmark output (`com.nearaid.benchmark-benchmarkData.json`). Coverage and startup are measured; build-time is measured but variable; nothing is inferred.